MEMORIA DEL PROYECTO INDIVIDUAL: DEAD SOULS

Módulo: Programación — 1.º DAW
Curso Académico: 2026/2027
Entrega: Primera Evaluación — Entrega 1
Alumno: Jorge Fernández Redondo
URL de mi GitHub: https://github.com/jfdez828/DeadSouls

1. DESCRIPCIÓN DEL JUEGO Y REGLAS DE PARTIDA

1.1. Temática y Objetivo

Dead Souls es un videojuego de rol (RPG) de corte clásico diseñado para ser ejecutado íntegramente a través de la consola de comandos. El jugador asume el rol de un aventurero que debe explorar un reino hostil infestado de criaturas.

El objetivo principal de la aventura consiste en **derrotar a un total de 10 enemigos comunes** en la opción de exploración para limpiar la región. Una vez alcanzada esta cifra exacta de bajas, se activa un evento ambiental ("el cielo se vuelve negro") que invoca inmediatamente al jefe final de la zona: **El Señor de las Sombras**. El propósito definitivo de la partida es vencer a este Boss en un combate singular para restaurar la paz en el reino.

1.2. Desarrollo de la Partida y Mecánicas del Juego

La ejecución del programa se rige bajo una serie de fases lógicas bien definidas:

1. **Inicialización y Creación del Personaje:** El programa solicita de forma interactiva el nombre del avatar del usuario. Acto seguido, proporciona un fondo de 10 puntos de estadísticas libres que el usuario debe repartir en su totalidad para moldear los atributos iniciales del héroe.
2. **Menú de Operaciones Principal:** Es el núcleo central de navegación. El usuario puede elegir libremente entre explorar para combatir, monitorizar su estado, gestionar consumibles e interactuar con el comercio local.
3. **Exploración y Combate:** Al adentrarse en el mapa, el sistema invoca de forma pseudoaleatoria a una de las amenazas de la base de datos. La confrontación se resuelve mediante un motor de combate por turnos estrictos donde el jugador toma decisiones tácticas y los adversarios responden de forma automatizada mediante cálculos algorítmicos.
4. **Ciclo de Recompensa y Progresión:** Cada victoria frente a un monstruo común nutre al héroe con recursos financieros (monedas) y experiencia acumulativa que propicia subidas automáticas de nivel.

1.3. Condiciones de Victoria y Derrota

- **Condición de Victoria Absoluta:** Se alcanza únicamente cuando el jugador consigue mermar por completo los puntos de vida (PV) del Boss Final (*El Señor de las Sombras*), activando un mensaje de enhorabuena y deteniendo la ejecución global del juego de forma exitosa.
- **Condición de Derrota (Fin de Partida Temporal con Penalización):** Si en cualquier momento del combate la salud actual del héroe llega a un valor

menor o igual a cero (vidaActual <= 0), este cae derrotado. El juego aplica penalizaciones automáticas y restaura parcialmente la salud a través de un rescate narrativo de los aldeanos, permitiendo continuar la aventura desde el menú principal pero con una importante merma de recursos.

2. FUNCIONALIDADES IMPLEMENTADAS Y DIFERENCIAS RESPECTO A LA PROPUESTA INICIAL

2.1. Funcionalidades del Sistema de Menús

El juego implementa una arquitectura modular de interacción basada en un menú raíz compuesto por 5 opciones operativas:

- **Opción 1: Explorar el Reino:** Controla tanto la selección determinista/aleatoria de enemigos como el bucle interno de combate por turnos. Altera directamente las propiedades de salud, economía y experiencia del personaje.
- **Opción 2: Consultar Estadísticas:** Muestra en pantalla el estado actual y los límites físicos del héroe (Nivel, Experiencia actual y necesaria, Puntos de vida reales/máximos, daño base, mitigación por defensa y balance de monedas).
- **Opción 3: Ver Inventario / Usar Objeto:** Renderiza por pantalla los elementos almacenados en las ranuras físicas del vector de inventario y gestiona de forma interactiva la activación de consumibles fuera del estado de combate.
- **Opción 4: Visitar la Tienda:** Despliega un submenú comercial para realizar transacciones de compra de consumibles o mejoras permanentes de equipamiento pasivo.
- **Opción 5: Salir del Juego:** Realiza el cierre controlado del flujo de ejecución del programa.

2.2. Diferencias y Evolución sobre la Propuesta Inicial

En la concepción original de la propuesta (recogida en los primeros bocetos de la memoria), el inventario iba a ser meramente ilustrativo y el combate se calcularía de forma plana. Durante el desarrollo de esta primera evaluación, se añadieron las siguientes implementaciones técnicas no planificadas:

- **Inventario Dinámico en Combate:** Se dotó al sistema de la capacidad de desplegar el inventario en mitad de una pelea (Opción 3 del combate), controlando mediante el uso de palabras clave la mutación de elementos y garantizando la cancelación segura del turno sin pérdida de acciones.

- **Mecánica de Guardia Completa:** Se añadió la opción "Defenderse" en el bucle de pelea, la cual duplica aritméticamente el valor de mitigación de daño del héroe frente al ataque inminente del oponente durante ese turno específico.
- **Mitigación por Rangos Variables:** Los daños provocados por el héroe y los enemigos dejaron de ser estáticos; se añadieron desviaciones aleatorias mediante el uso de random.nextInt() para aportar dinamismo y realismo a la experiencia jugable.

3. DISEÑO TÉCNICO Y ARQUITECTURA DEL CÓDIGO

3.1. Variables y Constantes

Se ha seguido escrupulosamente la convención *camelCase* para los identificadores de variables y el uso de mayúsculas sostenidas separadas por guiones bajos (*UPPER_SNAKE_CASE*) acopladas a la palabra clave final para definir constantes del sistema.

Identificador Técnico	Tipo de Datos	Naturaleza	Función dentro del Flujo del Programa
ENEMIGOS_PARA_BOSS	int	Constante	Umbral fijo (10) de victorias requeridas para desbloquear la fase final.
nombreHeroe	String	Variable	Almacena el nombre del jugador ingresado por consola.
vidaMaxima	int	Variable	Límite superior de puntos de vida del héroe; aumenta con stats o niveles.
vidaActual	int	Variable	Salud en tiempo real del personaje en menús y refriegas.
ataqueHeroe	int	Variable	Valor base utilizado para calcular la potencia ofensiva del jugador.
defensaHeroe	int	Variable	Valor base empleado para la reducción del daño bruto recibido.
puntosDisponibles	int	Variable	Contador regresivo de créditos asignables en la fase de creación.
nivel, experiencia	int	Variable	Parámetros que gobiernan el rango y progreso del jugador.
expSiguienteNivel	int	Variable	Techo de experiencia dinámica requerido para la siguiente subida de nivel.
monedas	int	Variable	Recursos monetarios disponibles para realizar transacciones comerciales.
enemigosDerrotados	int	Variable	Contador incremental acumulativo de victorias en el reino.
jugando, enCombate	boolean	Variable	Banderas de control de estados para la persistencia de bucles repetitivos.

3.2. Estructuras de Control

El programa articula el control de flujo basándose en las tres tipologías fundamentales de estructuras:

Bucles de Repetición (while / for)

- **while (jugando):** Gobierna el estado maestro de ejecución del juego, garantizando que el usuario regrese al menú principal al terminar una acción, salvo que se active la condición de salida voluntaria u otra de fin de partida definitivo.
- **while (puntosDisponibles > 0):** Bloquea el flujo inicial del programa. No se concede acceso al juego hasta que el jugador asigne equitativamente los 10 créditos a sus atributos.
- **while (enCombate):** Determina la persistencia de la interfaz de la batalla. Se repite de forma indefinida hasta que la salud de alguno de los dos contendientes descienda a cero.
- **for (int i = 0; i < inventario.length; i++):** Utilizado extensamente para el escaneo secuencial de arrays, permitiendo tanto listar elementos por pantalla como localizar índices disponibles con el literal "Vacio".

Condicionales Alternativos (if-else / switch)

- **Estructuras switch:** Implementadas para la canalización directa de decisiones numéricas explícitas, tales como la selección de opciones del menú principal (1 al 5), las ramas de mejora de atributos en la creación (1 al 3) y el catálogo de artículos de la tienda del reino (1 al 4).
- **Estructuras if-else anidadas:** Encargadas de evaluar condiciones complejas o rangos lógicos, por ejemplo, validar si el monedero del jugador cuenta con saldo suficiente frente al precio del artículo (monedas >= 20) o determinar si el contador de bajas da paso al Boss final.

Sentencias de Salto e Interrupción

- **break:** Empleado de forma obligatoria para evitar la caída en cascada en las cláusulas de los switch, así como para forzar la salida inmediata de los bucles de combate en el momento exacto en que ocurre un fallecimiento.
- **continue:** Integrado de forma estratégica dentro del inventario en combate. Si el jugador comete un error en la selección de un slot o decide pulsar la opción "Cancelar y volver", el programa interrumpe la iteración actual del turno y retrocede al menú de combate original mediante el refresco inmediato de la pantalla, previniendo que el enemigo ataque por un error del usuario.

3.3. Uso de Arrays Unidimensionales y Estructuras Paralelas

Debido a las restricciones técnicas de la primera evaluación, donde todavía no se contempla la Programación Orientada a Objetos (POO), se han implementado dos estrategias basadas en arrays nativos:

1. Vector de Inventario Lineal (String[])

Consiste en un array estático indexado de 5 posiciones que almacena cadenas de texto (String). El inventario comienza precargado con tres objetos fijos y dos posiciones disponibles representadas explícitamente mediante el literal "Vacio":

```java
String[] inventario = { "Pocion de vida", "Espada oxidada", "Escudo de madera", "Vacio", "Vacio" };
```

Usa el código con precaución.

Cuando el jugador compra un artículo consumible en la tienda, un bucle condicionado busca secuencialmente el primer índice que devuelva verdadero a .equals("Vacio"), mutando ese literal por el nombre del nuevo artículo. Al consumirse, el elemento vuelve a escribirse como "Vacio".

2. Base de Datos Síncrona mediante Arrays Paralelos

Para estructurar la fauna de monstruos comunes, se han definido tres arrays independientes de igual tamaño que comparten semánticamente la correspondencia de sus datos a través de sus índices posicionales:

```java
String[] enemigosNombres = { "Duende", "Lobo Salvaje", "Esqueleto", "Orco" };
int[] enemigosVidaMax = { 30, 45, 60, 80 };
int[] enemigosAtaque = { 8, 12, 15, 20 };
```

Usa el código con precaución.

- **Mecánica de sincronización:** El motor de exploración genera un índice aleatorio acotado: int indiceEnemigo = random.nextInt(enemigosNombres.length);.
- Si el número generado es el 2, el sistema extrae de forma síncrona el nombre del índice enemigosNombres[2] ("Esqueleto"), su salud de enemigosVidaMax[2] (60) y su poder de ataque de enemigosAtaque[2] (15). Esto simula la existencia de un objeto entidad sin salir del paradigma estructurado.

4. PLAN DE PRUEBAS Y CASOS LÍMITE

Para asegurar la robustez y resiliencia del software ante entradas anómalas o estados lógicos extremos, se ejecutó el siguiente plan de pruebas en consola:

4.1. Caso Incorrecto: Introducción de Caracteres Alfabéticos en Menús Numéricos

- **Descripción del peligro:** Cuando el programa espera una opción entera (p. ej., en el menú principal) y el usuario introduce texto plano (como "hola"), el método estándar de lectura arrojaría una excepción crítica de tipo InputMismatchException, provocando la caída del programa.
- **Solución y Comprobación en Código:** Se implementó el método personalizado leerEntero(Scanner scanner). Este método ejecuta un bucle de control basado en el predicado !scanner.hasNextInt(). Si detecta que el token en el buffer no es un entero válido, imprime un mensaje de aviso, consume y limpia la entrada incorrecta mediante scanner.next() y mantiene al usuario a la espera de un dato íntegro, salvaguardando la continuidad del hilo de ejecución.

4.2. Caso Límite: Gestión de Transacciones Comerciales sin Fondos Suficientes

- **Descripción de la prueba:** Acceder a la tienda del reino con el saldo base inicial de 50 monedas. Comprar un artículo pesado (p. ej., *Escudo reforzado* por 45 monedas). El saldo pasa a ser de 5 monedas. Acto seguido, intentar adquirir una *Poción de vida* (cuyo valor de mercado es de 20 monedas).
- **Resultado Obtenido:** El condicional de control if (monedas >= 20) evalúa la expresión 5 >= 20 como falsa, saltando de manera limpia hacia la cláusula else, donde se notifica explícitamente: *"No tienes suficientes monedas."*, bloqueando la asignación del objeto y manteniendo el estado financiero intacto.

4.3. Caso Límite: Desbordamiento de la Capacidad Física del Inventario

- **Descripción de la prueba:** Forzar al inventario a un estado de saturación completa comprando consumibles de forma sucesiva hasta rellenar los 5 slots del vector con elementos válidos eliminando cualquier rastro de la palabra "Vacio". Una vez lleno, intentar comprar una poción adicional en la tienda.
- **Resultado Obtenido:** El bucle for encargado del escaneo lineal evalúa los 5 índices sin encontrar ninguna coincidencia con la cadena "Vacio", lo que mantiene la variable centinela huecoVacio con su valor inicial de -1. Al concluir el escaneo, la estructura condicional evalúa si huecoVacio != -1; al resultar falso, deniega la compra, retiene el dinero del jugador e informa en pantalla: *"No tienes espacio en tu inventario."*.

5. DIFICULTADES ENCONTRADAS, SOLUCIONES Y AMPLIACIONES FUTURAS

5.1. Dificultades Técnicas Encontradas y Soluciones Aplicadas

1. Desincronización y Omisión de Captura de Cadenas por el Buffer de Scanner

- **El Problema:** Durante las primeras etapas del desarrollo, tras realizar lecturas numéricas mediante el método nativo `scanner.nextInt()`, el programa saltaba de forma abrupta e inexplicable la siguiente línea de instrucción donde se requería que el usuario introdujera una variable de texto (como el nombre del héroe).
- **La Causa:** El método `nextInt()` procesa únicamente los caracteres numéricos del flujo de entrada, dejando huérfano el carácter de escape de salto de línea (`\n`) dentro del buffer interno del stream de entrada. Al llegar al posterior `nextLine()`, este consumía el `\n` remanente interpretándolo como una pulsación de *Intro* en blanco.
- **La Solución:** Se encapsuló la limpieza de forma sistemática dentro del método auxiliar `leerEntero()`. Inmediatamente después de capturar el dato entero mediante `scanner.nextInt()`, se fuerza una llamada explícita a `scanner.nextLine()` para barrer y purgar cualquier residuo del salto de línea antes de devolver el flujo al método principal, garantizando lecturas de texto limpias en adelante.

2. Paradoja de Ataques Absorbidos y Efecto de Autocuración en Combate

- **El Problema:** Al dotar al jugador de la capacidad de adquirir equipamiento avanzado en la tienda (como el *Escudo reforzado*), se daban escenarios donde la estadística `defensaTotal` superaba numéricamente al daño de ataque bruto infligido por criaturas de bajo rango (como el *Duende*). Al aplicar la fórmula aritmética directa: `vidaActual -= (enemigoAtaque - defensaTotal)`, la operación resultaba en la resta de un valor negativo. Siguiendo las reglas algebraicas, restar un número negativo equivale a una suma (`- (-x) = +x`), haciendo que los golpes de los enemigos curaran al jugador.
- **La Solución:** Se erradicó este comportamiento anómalo mediante la introducción de la función matemática de acotamiento `Math.max(1, dañoCalculado)`. Esta función compara el resultado numérico del daño frente a un límite inferior rígido de `1`. De este modo, si la defensa del héroe anula por completo el ataque del monstruo, la función fuerza a que el daño neto final

aplicable sea siempre, como mínimo, de 1 punto de daño, preservando el peligro en el juego.

5.2. Ampliaciones Planificadas para la Segunda Evaluación (Proyecto Incremental)

Al concebirse este proyecto como una base incremental para las entregas venideras, se tienen proyectadas las siguientes mejoras arquitectónicas:

1. **Migración Completa al Paradigma de Programación Orientada a Objetos (POO):** Sustituir de manera definitiva el sistema rudimentario de arrays paralelos mediante la declaración de clases formales con un alto nivel de encapsulamiento (Heroe, Enemigo, Item). Esto permitirá dotar a los entes de constructores propios, métodos de acción internos (como .atacar()) y propiedades privadas protegidas por *Getters* y *Setters*.
2. **Adopción de Estructuras Dinámicas mediante la API de Colecciones (ArrayList):** Reemplazar el uso de arrays nativos de tamaño estático para el inventario por colecciones dinámicas de tipo ArrayList<Item>. Esta ampliación eliminará el límite estricto de 5 posiciones fijas, facilitando la inserción, descarte y ordenación de objetos mediante los métodos nativos .add() y .remove().
3. **Persistencia de Datos en Ficheros de Disco:** Desarrollar un módulo de entrada/salida (I/O) enfocado en la lectura y escritura de archivos de texto planos (.txt) o serializados. Esto permitirá guardar el estado exacto de las variables del jugador al salir de la aplicación, haciendo viable la persistencia y carga de partidas guardadas en ejecuciones futuras.